

Univerzita Karlova

Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země



Aliaksandra Sparysh

Jolana Václavková

Předmět: Geoinformatika

Nejkratší cesta grafem

Zadání

Implementujte Dijkstra algoritmus pro nalezení nejkratší cesty mezi dvěma zadanými uzly grafu.

Vstupní data budou představována silniční sítí doplněnou vybranými sídly (alespoň 100 uzlů). S využitím přiloženého skriptu konvertujte podkladová data do grafové reprezentace.

Otestujte různé varianty volby ohodnocení w hran grafu tak, aby nalezená cesta měla:

- nejkratší Eukleidovskou vzdálenost,
- nejmenší transportní čas (2 varianty, bez/se zohledněním klikatosti komunikací).

Pro druhou variantu optimální cesty navrhnete také vhodnou metriku, která zohledňuje rozdílnou dobu jízdy na různých typech komunikací dle jejich návrhové rychlosti v a klikatosti κ , příkladem může být

$$t(u, v) = \kappa \frac{l(u, v)}{v(u, v)}.$$

Pro výpočet κ využijte poměr délky polylinie l představující diskretní křivku a vzdálenosti s koncových bodů polylinie

$$\kappa = \frac{l(u, v)}{s(u, v)}.$$

Každou z variant otestujte pro dvě dvě různé cesty tvořené alespoň 20 uzly. Výsledky umístěte do tabulky, vlastní cesty vizualizujte. Dosažené výsledky porovnejte s vybraným navigačním SW (alespoň 3).

Kromě povinného Dijkstrova algoritmu byly konstruovány tyto bonusové úlohy (vyznačeny kurzívou):

Dijkstra algoritmus	20 bodů
<i>Návrh jiného ohodnocení hran zohledňující křivolakost silniční sítě.</i>	<i>5 bodů</i>
<i>Zohlednění vlivu DMT.</i>	<i>15 bodů</i>
<i>Nalezení minimální kostry Borůvka/Kruskal.</i>	<i>15 bodů</i>

Teoretická část

Cílem této kapitoly je nejprve všechny zadané úlohy, resp. jejich principy teoreticky popsat.

Dijkstrův algoritmus, který v roce 1959 zformuloval Edsger Dijkstra, je dnes považován za nejrozšířenější metodu pro hledání nejkratších cest v orientovaných grafech s nezáporně ohodnocenými hranami (Valla a Mareš, 2017). Praktické fungování algoritmu připomíná systém „budíků“. Pro každý vrchol, ke kterému směřuje vlna šířená z počátečního bodu v_0 , se nastaví časový údaj odpovídající okamžiku, kdy do něj vlna má dorazit poprvé. Algoritmus pak v každém kroku „přeskočí“ k nejbližšímu nastavenému budíku, tedy k vrcholu s aktuálně nejmenší dosaženou vzdáleností. Po „zazvonění“ budíku se prozkoumají všechny hrany vycházející z daného vrcholu a případně se přenastaví budíky v sousedních vrcholech na dřívější čas, pokud byla nalezena výhodnější cesta (Valla a Mareš, 2017). Tento proces se odborně nazývá relaxace hran a formálně jej lze zapsat podmínkou:

$$h(w) > h(v) + l(v, w)$$

kde jednotlivé proměnné znamenají:

$h(w)$ - Dosud nalezená nejkratší vzdálenost od startu do sousedního vrcholu w .

$h(v)$ - Vzdálenost do právě uzavíraného vrcholu v , která je již v tuto chvíli optimální.

$l(v, w)$ - Ohodnocení (délka) hrany vedoucí přímo z vrcholu v do vrcholu w .

Pokud je tato podmínka splněna, hodnota $h(w)$ se aktualizuje na součet $h(v) + l(v, w)$.

V průběhu výpočtu algoritmus pracuje se třemi stavy vrcholů: nenalezené, otevřené a uzavřené. Z hlediska datových struktur algoritmus efektivně využívá pole předchůdců $P(v)$, které slouží k rekonstrukci samotné trasy. Pro libovolný uzel w lze výslednou nejkratší cestu získat zpětným procházením přes hodnoty $P(w)$, $P(P(w))$ až k výchozímu bodu v_0 .

Schéma Dijkstrova algoritmu

Algoritmus DIJKSTRA (nejkratší cesty v ohodnoceném grafu)

Vstup: Graf G a počáteční vrchol v_0

1. Pro všechny vrcholy v :
2. $stav(v) \leftarrow \text{nenalezený}$
3. $h(v) \leftarrow +\infty$
4. $P(v) \leftarrow \text{nedefinováno}$
5. $stav(v_0) \leftarrow \text{otevřený}$
6. $h(v_0) \leftarrow 0$
7. Dokud existují nějaké otevřené vrcholy:
8. Vybereme otevřený vrchol v , jehož $h(v)$ je nejmenší.
9. Pro všechny následníky w vrcholu v :
10. Pokud $h(w) > h(v) + \ell(v, w)$:
11. $h(w) \leftarrow h(v) + \ell(v, w)$
12. $stav(w) \leftarrow \text{otevřený}$
13. $P(w) \leftarrow v$
14. $stav(v) \leftarrow \text{uzavřený}$

Výstup: Pole vzdáleností h , pole předchůdců P

Zdroj: Mareš, Valla (2017)

V reálných dopravních sítích není délka hrany v metrech jediným faktorem ovlivňujícím efektivitu trasy. Parametr křivolakosti zohledňuje horizontální členitost silnice, tedy výskyt zatáček, které nutí řidiče zpomalovat. Teoreticky lze toto ohodnocení uchopit jako zavedení váhové funkce $w(e)$, která k fyzické délce připočítává penalizaci za "klikatost". Tu můžeme vyjádřit například pomocí koeficientu k :

$$k = \frac{l_{\text{skutečná}}}{l_{\text{vzdušná}}}$$

Nová váha hrany tedy poté vypadá takto:

$$w(e) = l_{\text{skutečná}} \cdot k$$

Tento přístup transformuje geometrickou složitost trasy do abstraktní ceny hrany. Cesta, která je sice geometricky kratší, ale obsahuje velké množství ostrých zatáček, tak získá vyšší celkové ohodnocení než cesta delší, ale přímá.

Integrace Digitálního modelu terénu (DMT) do grafových úloh rozšiřuje výpočet o vertikální dimenzi. Vliv terénu se do váhy hrany promítá skrze gradient (sklon) s mezi dvěma body o nadmořských výškách z_1 a z_2 :

$$s = \frac{z_2 - z_1}{l_{\text{horizontální}}}$$

Váha hrany $l(v, w)$ se vlivem DMT stává asymetrickou, protože cena za překonání úseku směrem nahoru je odlišná od směru dolů. Celkové ohodnocení můžeme modelovat jako:

$$l_{DMT}(v, w) = l(v, w) \cdot (1 + \phi(s))$$

Kde $\phi(s)$ je funkce penalizující stoupání. Tento model mění výsledný strom nejkratších cest a odklání trasu do oblastí s menším převýšením.

Posledním teoretickým konceptem je pak Borůvkův algoritmus – metoda pro nalezení minimální kostry grafu (tedy nejlevnějšího způsobu, jak propojit všechny body v síti). Funguje na principu postupného propojování částí grafu, které zatím nejsou spojené. Na začátku je každý bod sítě samostatnou komponentou. V každém kroku algoritmu si každá komponenta najde svou nejlevnější cestu k nejbližšímu sousední komponentě a spojí se s ní.

Tento proces probíhá v iteracích: v každém kole se počet samostatných částí výrazně zmenší, protože se spojují do větších celků. Celý výpočet končí ve chvíli, kdy zbude pouze jeden velký celek, který obsahuje všechny body sítě bez zbytečných smyček. Pro efektivní správu

těchto skupin se v kódu používá technika Union-Find, která si jednoduše pamatuje, který bod patří ke které skupině a jak je správně sloučit.

Data

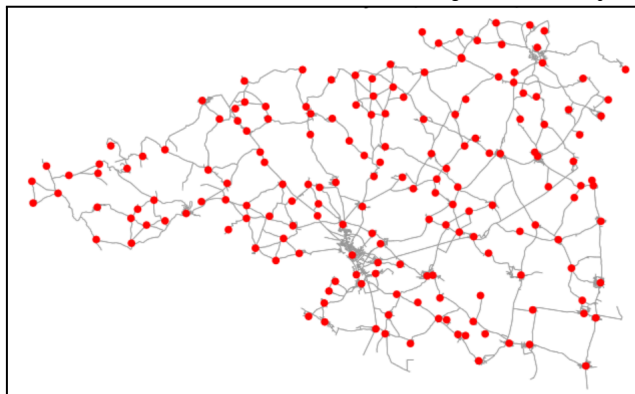
Jako vstupní data byla zvolena reálná silniční síť okresu Znojmo, čerpaná z otevřené databáze OpenStreetMap (OSM) prostřednictvím Python knihovny OSMnx. Data reprezentují komunikace sjízdné pro automobily, které byly modelovány jako neorientovaný vážený graf, kde uzly představují křižovatky a hrany odpovídají jednotlivým silničním úsekům, jejichž váhou je jejich fyzická délka. Druhou součástí datového vstupu je rovněž 173 sídel, která byla pro účely síťové analýzy z původních plošných polygonů redukována na geometrické středy (centroidy) a následně metodou nejbližšího souseda spárována s odpovídajícími uzly silniční sítě. Pro potřeby navazujícího algoritmického zpracování byla topologie exportována do zjednodušeného textového formátu obsahujícího kartézské souřadnice uzlů.

Pro převod vstupních dat na funkční model sítě program nejprve načte textový soubor obsahující souřadnice a délky jednotlivých silničních úseků. Klíčovým krokem je následné sjednocení křižovatek, kdy algoritmus rozpozná, že silnice končící na shodných GPS souřadnicích se fakticky potkávají v jednom bodě, a přidělí tomuto bodu unikátní identifikační číslo. Tímto procesem se jednotlivé úseky logicky propojí, čímž vzniká ucelená struktura tvořící výslednou grafovou reprezentaci silniční sítě, již lze vidět na Obr. 1.

V rámci práce je použit generátor náhodných hodnot, (slouží k ověření, že navržený algoritmus je skutečně aplikovatelný na celý graf a není přizpůsoben pouze konkrétním testovacím případům).

Pro použití stejných testovacích tras při opakování je náhodně upraveno pomocí `random.seed(10)`. Vizualizace trasy je v této práci zobrazena pouze jednou, jelikož ve v uvažovaném případech vychází všechny shodně.

Obr. 1: Silniční síť okresu Znojmo se sídly



Implementace

1. Dijkstra algoritmus

a. nejkratší Euklidovská vzdálenost

Jako metrika je použita druhá mocnina euklidovské vzdálenosti definovaná funkcí `distance_squared(x1, y1, x2, y2)`. Tento přístup je preferovaný z důvodu výpočetní efektivity, jelikož eliminuje potřebu výpočtu odmocniny. Metrika sice neodpovídá přímé fyzické délce, ale zachovává poměry pro hledání nejkratší cesty.

Souřadnice jednotlivých bodů jsou převedeny na unikátní celočíselné identifikátory pomocí funkce `souradnice_na_id(body)`, což je potřeba pro použití grafových algoritmů. Dále je sestaven neorientovaný graf pomocí funkce `hrany_do_grafu(slovník_id, start_points, end_points, weights)`. Každá hrana je definována dvojicí uzlů a příslušnou vahou, přičemž graf je uložen ve formě slovníku.

Pro nalezení nejkratší cesty mezi zadaným počátečním a cílovým uzlem je použit Dijkstrův algoritmus, implementovaný funkcí `dijkstra(graf, start, end)`. Algoritmus pracuje s prioritní frontou, která zajišťuje, že v každém kroku je k dalšímu zpracování vybrán uzel s nejmenší známou vzdáleností od počátečního bodu.

Uvnitř cyklu následně probíhá relaxace hran – u každého souseda je matematicky ověřováno, zda k němu nevede výhodnější trasa právě přes aktuální bod (zkratka). Pokud je nalezena výhodnější cesta, je aktualizována nejkratší vzdálenost k danému uzlu a zároveň je uložen odkaz na jeho předchůdce, což umožňuje zpětně rekonstruovat celou výslednou trasu od cíle až na start.

Funkčnost řešení byla ověřena na testovacích datech při hledání cesty mezi uzlem 0 a 9. Algoritmus úspěšně našel trasu vedoucí přes sekvenci bodů [0, 3453, 1645, 3084, ..., 4253, 2888, 9] s celkovou vypočtenou vahou (po zaokrouhlení) 0.005298.

b. nejmenší transportní čas bez zohlednění klikatosti

V rámci předzpracování dat je graf silniční sítě transformován na časově vážený model. V základním nastavení funkce (`consider_curvature = False`) je cestovní čas každé hrany stanoven výhradně na základě její délky a odhadované průměrné rychlosti. Nejprve je aplikován rychlostní profil, který různým kategoriím komunikací přiřazuje návrhové rychlosti v rozmezí 30 až 90 km/h. Výsledný čas je následně vypočten jako podíl fyzické délky daného úseku a této rychlosti.

Pro výpočet optimální trasy je použit opět Dijkstrův algoritmus, který v každém kroku vybírá uzel s nejmenší známou hodnotou kumulovaného transportního času. V průběhu algoritmu dochází k relaxaci hran, při níž je testováno, zda cesta vedená přes aktuální uzel neposkytuje kratší časovou alternativu oproti dosud nalezené trase.

Při samotném výpočtu Dijkstrova algoritmu, je stanovena podmínka, že pokud délka již nalezené cesty přesáhne stanovený limit (20 uzlů), je tato cesta vrácena jako výsledek, jinak je znovu spuštěn výpočet nejkratší trasy s využitím časové váhy.

V tomto nastavení model tedy abstrahuje od zakřivení úseku a předpokládá konstantní průjezdní rychlost po celé délce hrany.

c. nejmenší transportní čas se zohledněním klikatosti

V této části je rozšířen základní model času o vliv klikatosti silničních úseků. Reálná doba průjezdu komunikací není dána jen délkou a návrhovou rychlostí, ale je ovlivněna také zakřivením trasy (pomocí `consider_curvature = True`). Základní časová váha hrany je definována jako podíl její délky a přiřazené návrhové rychlosti.

Klikatosti využívá prostorové souřadnice koncových uzlů hrany k určení jejich vzájemné vzdálenosti „vzdušnou čarou“. Klikatost úseku je následně vyjádřena poměrem mezi skutečnou délkou komunikace a touto přímkou vzdáleností. Čím více se silnice odchyluje od přímého směru, tím vyšší je hodnota tohoto faktoru. Základní cestovní čas je pak vynásoben tímto koeficientem, čímž dochází k penalizaci klikatých úseků. Výsledkem je časová váha, která reflektuje i geometrickou složitost komunikace. Tyto váhy jsou uloženy do nového grafu, jenž je následně použit jako vstup pro algoritmus hledání nejkratší cesty.

Vyhledání optimální trasy je znovu realizováno pomocí Dijkstrova algoritmu aplikovaného na graf s touto modifikovanou časovou vahou hran. Výsledkem je trasa minimalizující celkový penalizovaný transportní čas. První testovaná cesta vede z obce Znojmo do obce Hnanice. Druhou testovanou cestou je trasa Šafov - Černín. Trénovací cesty jsou vyobrazeny na Obr. 2 a 3. Porovnání metod je poté zobrazeno v Tab. 1.

2. Návrh jiného ohodnocení hran zohledňující křivolakost silniční sítě

V rámci této úlohy byl implementován algoritmus, který rozšiřuje standardní vyhledávání trasy o faktor geometrické efektivity. Program nehodnotí jednotlivé úseky pouze podle času průjezdu, ale modifikuje jejich váhu na základě toho, jak moc se odchylují od přímého směru. Klíčovým detailem v kódu je výpočet poměru mezi skutečnou délkou silnice a její vzdušnou

vzdáleností, který se následně umocňuje na druhou. Tato kvadratická závislost zajišťuje, že zakřivení nezvyšuje náročnost trasy lineárně, ale progresivně – čím je silnice klikatější, tím výrazněji její váha v grafu narůstá.

3. Zohlednění vlivu DMT

Původní algoritmus byl v další fázi rozšířen o zpracování digitálního modelu terénu, který umožňuje měnit rychlost podle profilu trasy. Skript pro každý úsek silnice vypočítá jeho sklon a podle něj aplikuje rychlostní pravidla. Model rozlišuje několik variant – v případě prudkého stoupání (sklon nad 5 %) snižuje základní rychlost o 30 %, u mírnějších kopců pak aplikuje 10% zpomalení. Naopak při jízdě z kopce algoritmus simuluje zrychlení a navyšuje rychlost o 10 až 30 %. Aby však simulace zůstala v mezích reality, je výsledná rychlost omezena pevnými hodnotami (minimální a maximální povolenou rychlostí), což zabraňuje výpočtu nereálně rychlých průjezdů. Teprve tato terénem a sklonem upravená rychlost následně vstupuje do výpočtu penalizace za zakřivení.

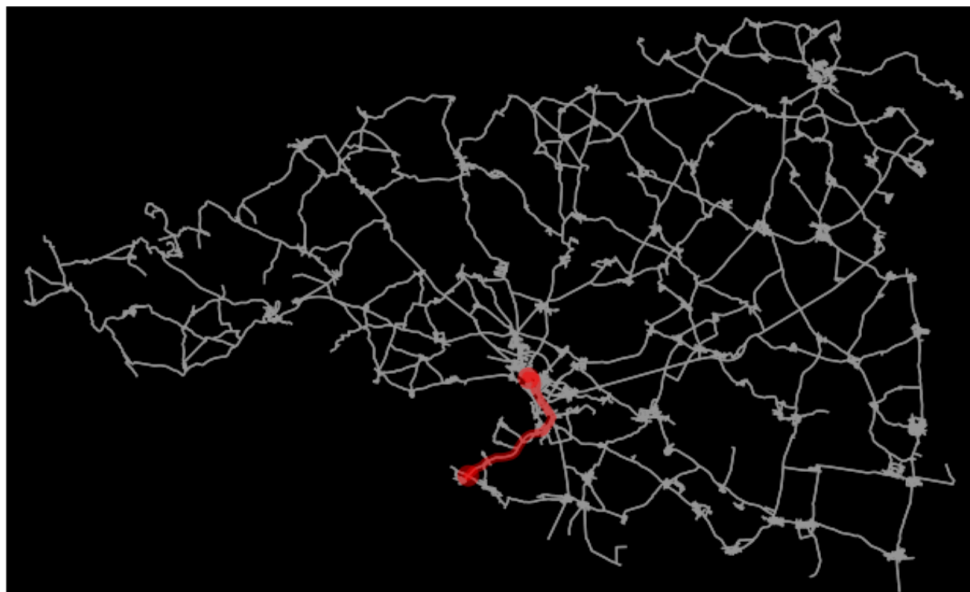
4. Nalezení minimální kostry Borůvka

Borůvkův algoritmus využívá pro správu souvislých částí grafu princip disjunktních množin (Union-Find), který je v kódu realizován funkcemi `init_komponenty`, `najdi` a `spoj`. Na počátku je každá křižovatka považována za izolovanou komponentu a v každé fázi se pro ni nalezne nejkratší možná hrana vedoucí k jiné části sítě. Takto vybrané úseky jsou zařazeny do výsledné minimální kostry a příslušné komponenty se následně sloučí do větších celků. Proces se opakuje, dokud není celá síť kompletně propojena.

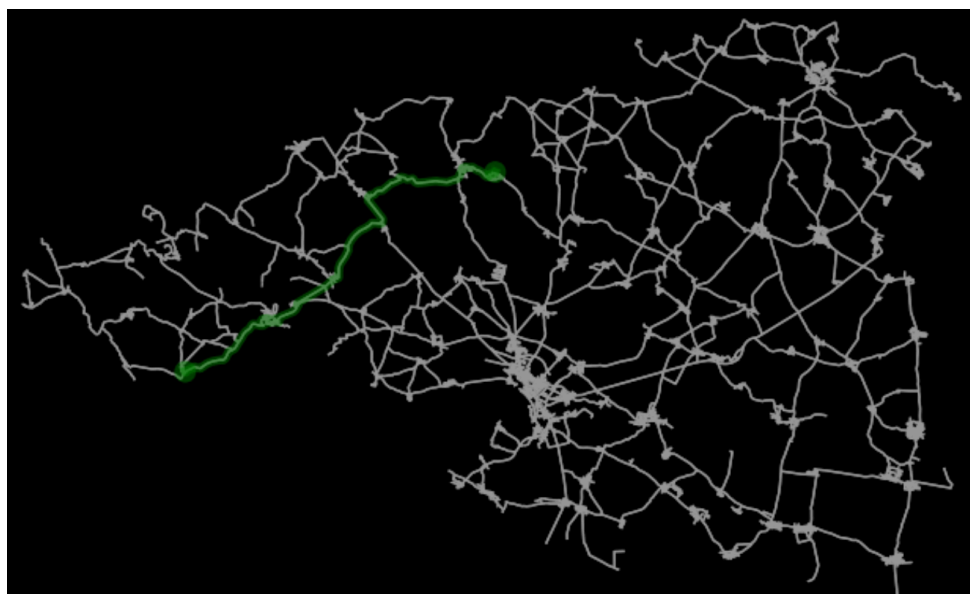
Přehled výsledků

Na Obr. 2 a na Obr. 3 jsou zobrazeny testované trasy. Ty se i přes změny parametrů nezměnily.

Obr. 2: První testovaná trasa z města Znojmo do obce Hnanic



Obr. 3: Druhá testovaná trasa ze sídla Šafov do obce Černín



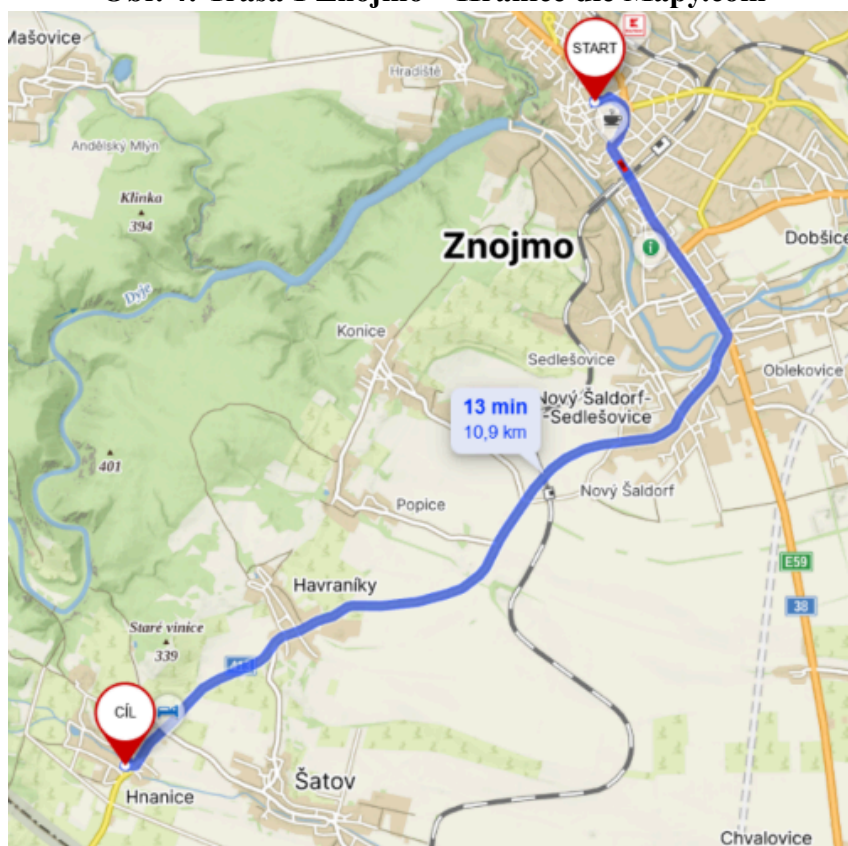
Předložená Tab. 1 srovnává časovou náročnost dvou tras ve čtyřech různých variantách ohodnocení hran silniční sítě. Trasa 1 (Znojmo–Hnanice) má základní čas stanoven na 558,06 s (9,3 min), přičemž se započtením koeficientu čas vzrůstá na 569,04 s (9,5 min) a u jiného ohodnocení na 580,56 s (9,7 min). Nejpomalejší variantou u této trasy je model DTM s časem 595,9 s, což odpovídá téměř 10 minutám. Trasa 2 (Šafov–Černín) začíná na základním čase 1694,3 s (28,2 min), který se s koeficientem zvyšuje na 1874,19 s (31,2 min) a při jiném ohodnocení dosahuje maxima 2093,32 s (34,9 min). U varianty DTM u této trasy dochází k mírnému poklesu na 2049,83 s, což představuje výsledný čas přibližně 34,2 minuty.

Tab. 1

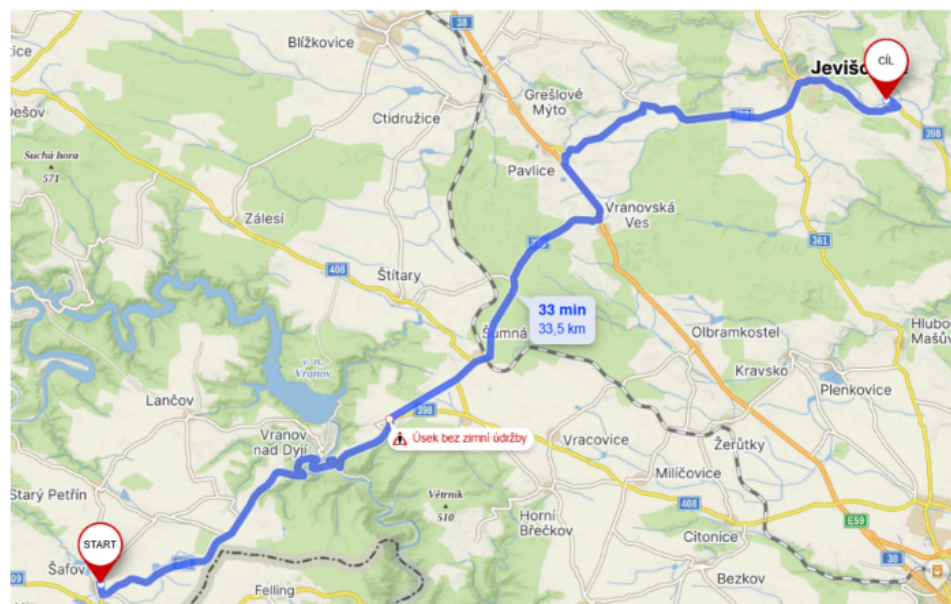
Trasa	Čas bez k. (s)	Čas s k. (s)	Čas jiného ohodnocení (s)	Čas DTM (s)
1	558.06	569.04	580.56	595.9
2	1694.3	1874.19	2093.32	2049.83

Dalším cílem tohoto úkolu je porovnat námi implementovaný algoritmus s jiným navigačním nástrojem. Rozhodly jsme se pro komparaci vlastní implementace DMT s [Mapy.com](https://www.mapy.com) (Tab. 4). Jak lze vidět, naše implementace v obou případech určila kratší časový interval. To však může být způsobeno tím, že [Mapy.com](https://www.mapy.com) ve svých výpočtech reflektují aktuální dopravní situaci (zdržení, opravy, semaforey – na Obr. 5 lze vidět na trase poznámku “bez údržby“), a právě proto nelze výsledky považovat za zcela srovnatelné. Časové rozdíly jsou pak ilustrovány v Tab. 2.

Obr. 4: Trasa 1 Znojmo – Hranice dle Mapy.com



Obr. 5: Trasa 2 Šafov – Černín



Tab. 2: Porovnání vlastní implementaci s DMT s Mapy.com

	Trasa 1	Trasa 2
Mapy.com	13 min	33 min
Djiskra algoritmus s DMT	10 min	34 min

Aplikací Borůvkova algoritmu na zpracovávaná data pro okres Znojmo byla získána optimální páteří síť tvořená 5 733 silničními úseky. Celková délka této minimální kostry grafu činí 929,267 km, což tedy, jak již bylo zmíněno, představuje matematicky nejmenší možnou vzdálenost nezbytnou pro vzájemné propojení všech uzlů v síti. Výsledky jsou znázorněny na Obr. 6.

Obr. 6

Number od edges in MST: 5733
Whole lenght (Eukleid): 929267.6675676835

Závěr

Předložená práce se zaměřila na aplikaci grafových algoritmů na reálné silniční síti okresu Znojmo s využitím dat z databáze OpenStreetMap. V rámci řešení byl implementován Dijkstrův algoritmus pro vyhledání nejkratších cest, který byl postupně rozšířen o penalizaci

na základě křivolakosti úseků a vlivu digitálního modelu terénu na cestovní rychlost. Srovnání s daty z Mapy.com potvrdilo, že zvolený výpočetní model generuje výsledky, které se blíží reálným časům v terénu. Součástí práce bylo rovněž nalezení minimální kostry grafu pomocí Borůvkova algoritmu, čímž byl definován matematicky nejkratší možný způsob propojení všech uzlů v síti.

Seznam literatury

BAYER, T.(2025): Předmět Geoinformatika, Přf UK. Přednáška 9: Grafové algoritmy.

BAYER, T.(2025): Předmět Geoinformatika, Přf UK. Přednáška 10: Nejkratší cesty v grafu.

JAVOID, A. (2013): Understanding Dijkstra's Algorithm. Institute of Electrical and Electronics Engineers (IEEE).

MAREŠ, M., VALLA, T. (2017): Průvodce labyrintem algoritmů. Edice CZ.NIC. 1. Vydání.